

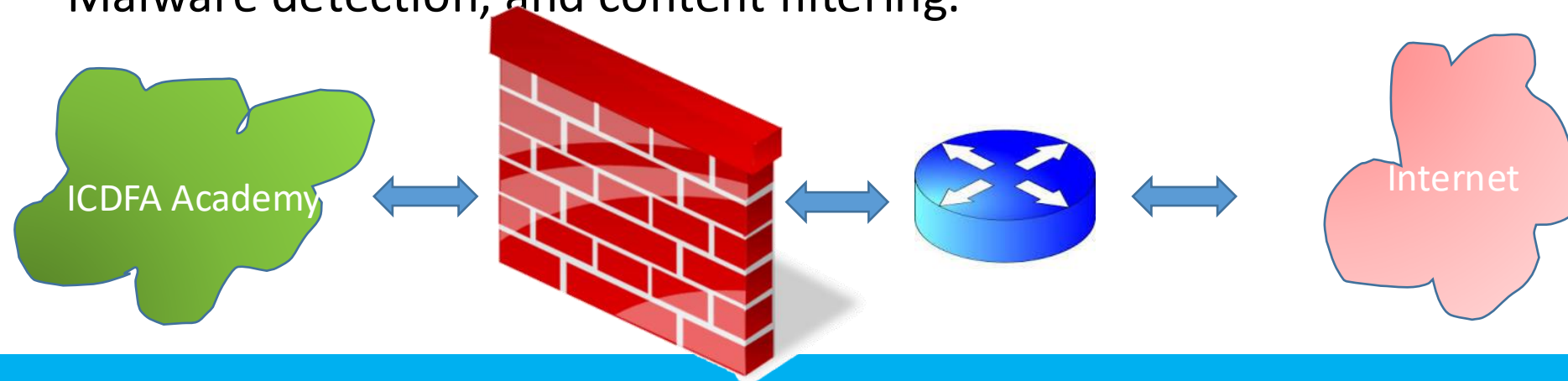
Firewall

Files: [firewall](#)



What is Firewall

- A firewall is a network security device that acts as a barrier between a private network and the public internet or other networks.
- Its primary purpose is to monitor and control incoming and outgoing network traffic based on predetermined security rules.
 - Intrusion detection and prevention
 - Malware detection, and content filtering.



The importance of a firewall

- Protect the network from unauthorized access, data breaches, and other security threats.
- Approach
 - Blocking or allowing traffic based on specific criteria, such as IP addresses, ports, protocols, and applications



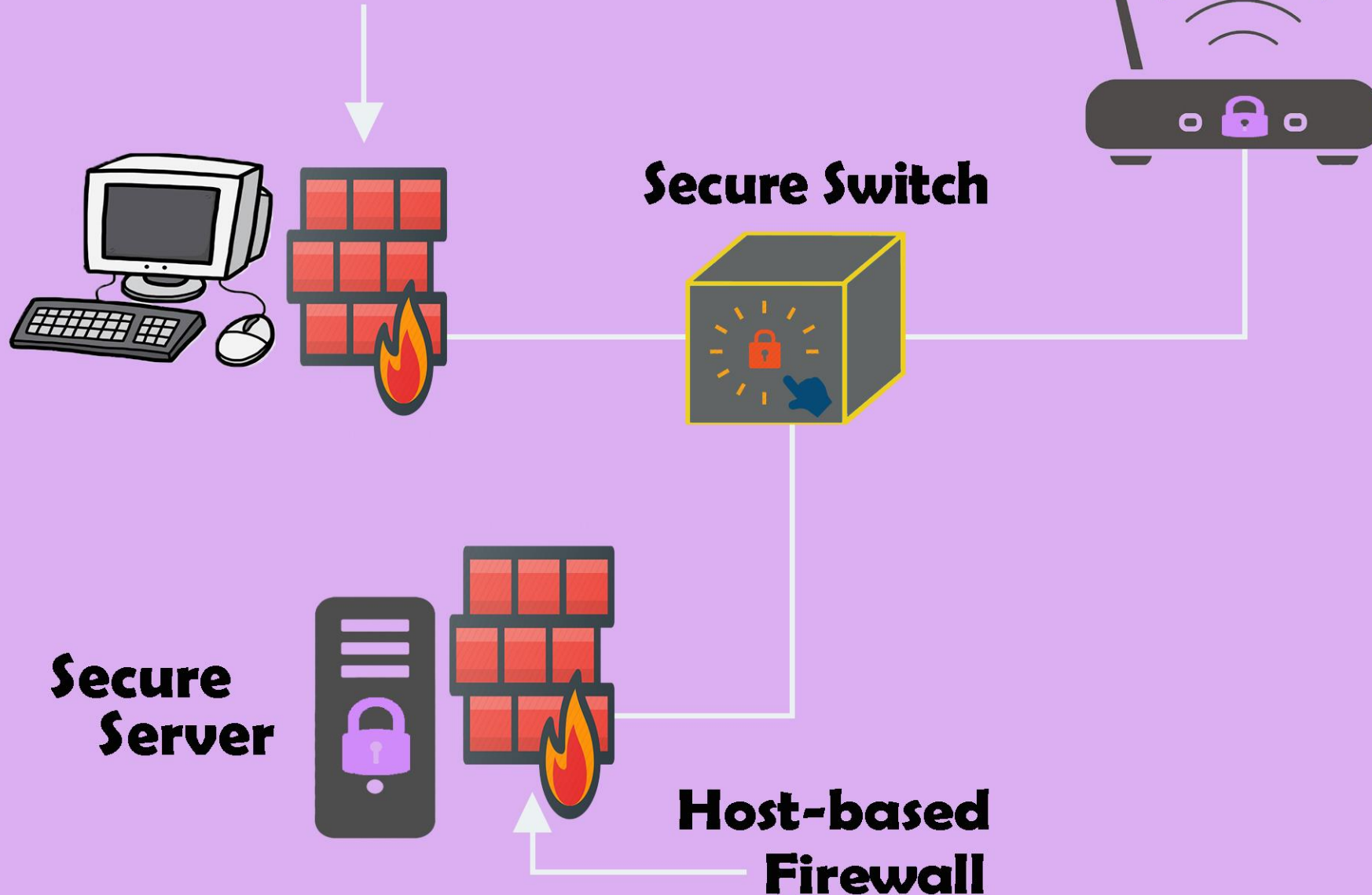
Firewalls Deploying Methods

- Network-based firewall
 - A device that sits between the internal network and the external network (usually the internet) and filters traffic based on predefined rules.
 - It provides a centralized security solution for the entire network and can control traffic from multiple hosts.
 - Network-based firewalls are effective at protecting against external threats, such as hackers and malware, but they may not be as effective at controlling internal traffic.
- Host-based firewall
 - installed on individual hosts (such as servers or workstations) and filters traffic on a per-host basis.
 - It provides a more granular level of control and can be customized for each host's specific needs.
 - Host-based firewalls are effective at protecting against internal threats, such as unauthorized access by insiders or malware that has already infected a host, but they may not be as effective at protecting against external threats.



Host-based Firewall

Secure Router

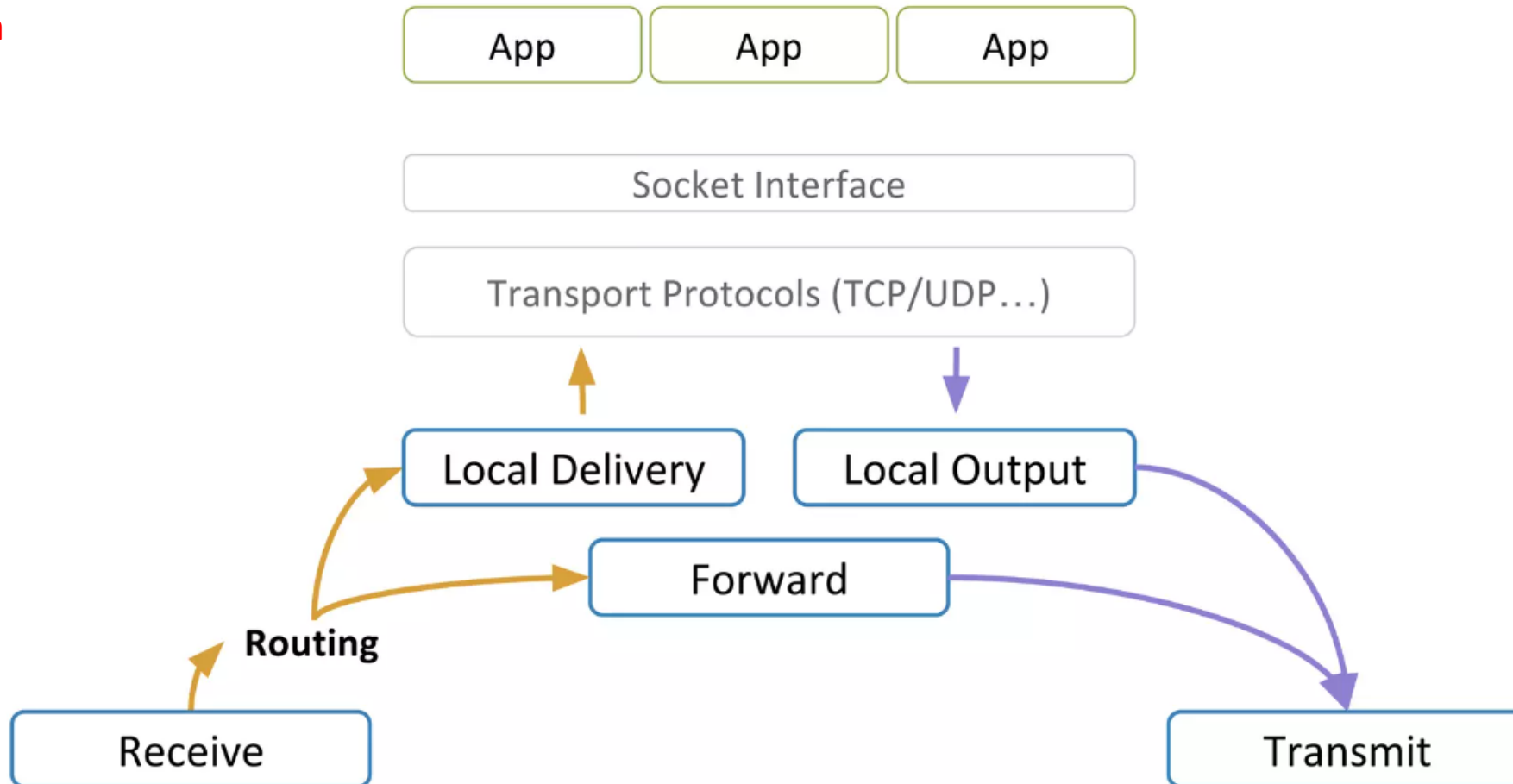


Linux Firewall (iptables)

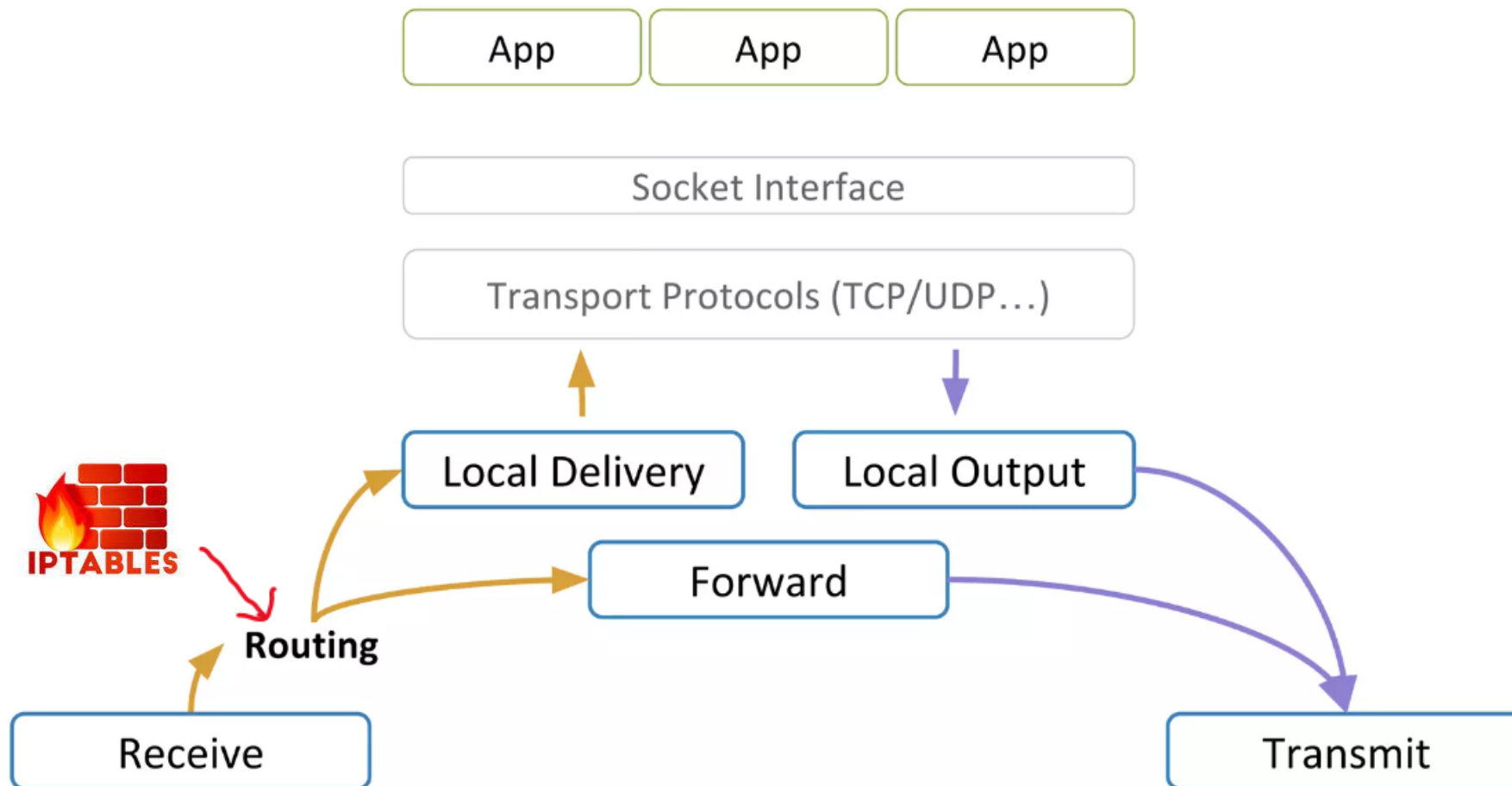


Host-based Routing Process

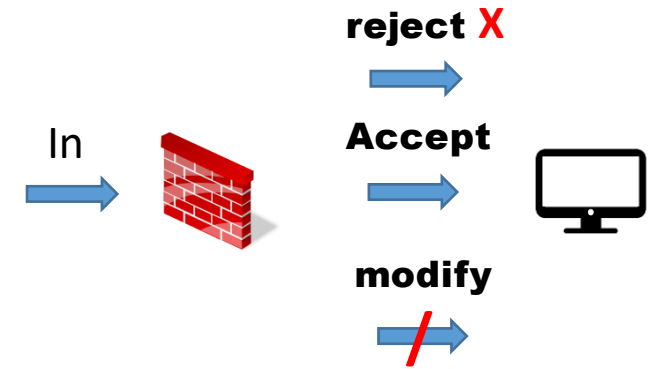
We focus on



Host-based Firewall



Linux Firewall and `iptables`



- `iptables` is a software firewall
 - pre-installed on most Linux-based operating systems.
- Linux Firewall defines a set of rules that determine how network traffic is allowed or blocked on a system
 - These rules are organized into chains, which are lists of rules that are evaluated in sequence.
- Each rule can match packets based on various criteria
 - such as the source or destination IP address, the protocol, and the port number.
- When a rule matches a packet, it can perform an action
 - such as accepting, rejecting, or modifying the packet.
- The "`iptables`" command-line utility is used to manage the firewall rules
 - it provides a powerful and flexible way to control network traffic on a Linux system.

iptables vs. ufw

- UFW (Uncomplicated Firewall) is a front-end tool for managing iptables firewall rules on a Linux system.
- UFW does not have its own set of rules separate from iptables
 - it translates the rules into iptables commands and applies them to the firewall.
 - it provides a simplified interface to manage iptables, which is a command-line utility for configuring netfilter firewall rules in the Linux kernel.

*disable ufw to
void conflits
with iptables
before the lab*



```
(student@kalit01)-[~/dns]
```

```
$ sudo ufw disable
```

```
Firewall stopped and disabled on system startup
```

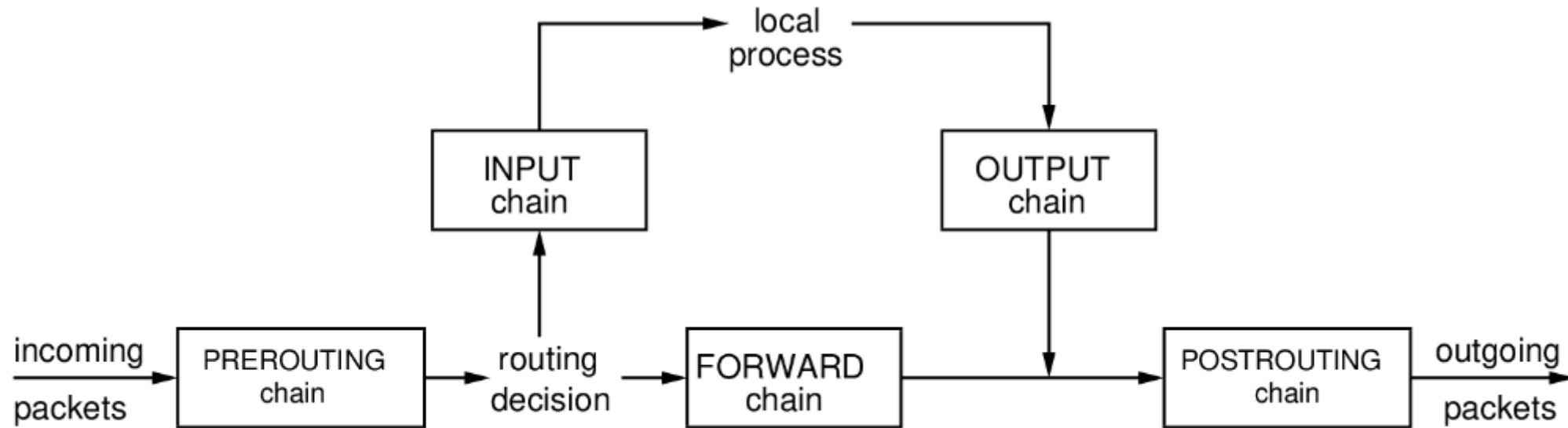
```
(student@kalit01)-[~/dns]
```

```
$ sudo ufw status
```

```
Status: inactive
```



Chains in iptables



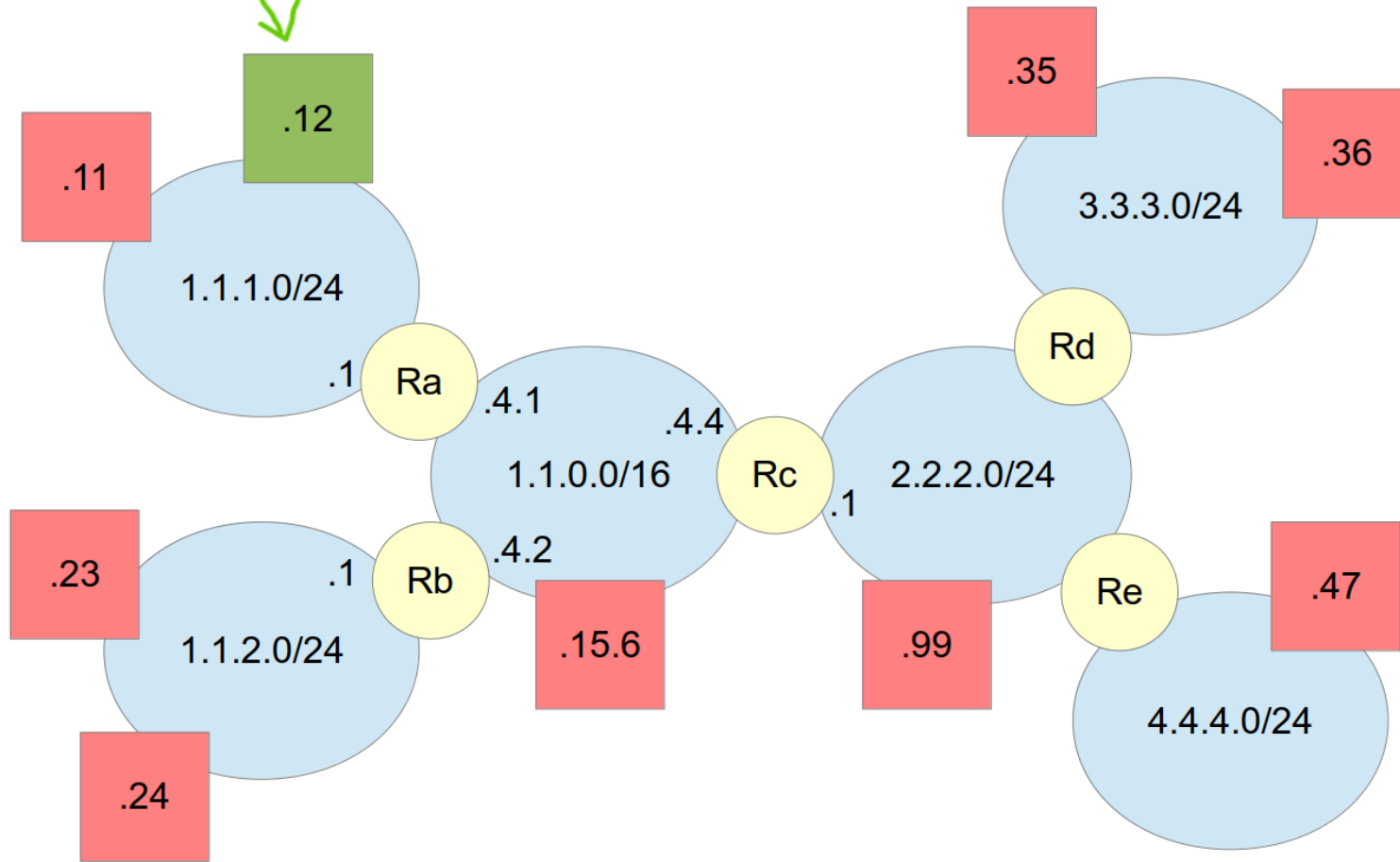
*iptables defines classes of **rules** called chains). Rules from each chain are applied based on where the packet is from/going to*

Rules in `iptables`

- Packet match conditions:
 - IP source address
 - IP destination address
 - TCP/UDP source port number
 - TCP/UDP destination port number
 - Other IP/TCP/UDP header fields
- Direction of traffic:
 - Is the packet coming from outside (to inside) or is it coming from inside (to outside)
- Actions:
 - ACCEPT or DROP



Host-Based Firewall running on 1.1.1.12



Add rules to iptables



```
(student@kalit01)-[~]  
$ mkdir dns  
  
(student@kalit01)-[~]  
$ cd dns
```

Show current rules in iptables

```
(student@kalit01)-[~/dns]  
$ sudo iptables -L --line-numbers
```

Input/forward/output chains

Chain INPUT (policy DROP)

num	target	prot	opt	source	destination
1	ufw-before-logging-input	all	--	anywhere	anywhere
2	ufw-before-input	all	--	anywhere	anywhere
3	ufw-after-input	all	--	anywhere	anywhere
4	ufw-after-logging-input	all	--	anywhere	anywhere
5	ufw-reject-input	all	--	anywhere	anywhere
6	ufw-track-input	all	--	anywhere	anywhere

Chain FORWARD (policy DROP)

num	target	prot	opt	source	destination
1	ufw-before-logging-forward	all	--	anywhere	anywhere
2	ufw-before-forward	all	--	anywhere	anywhere
3	ufw-after-forward	all	--	anywhere	anywhere
4	ufw-after-logging-forward	all	--	anywhere	anywhere
5	ufw-reject-forward	all	--	anywhere	anywhere
6	ufw-track-forward	all	--	anywhere	anywhere


```

(student@kalit01)-[~/dns]
$ iptables --help
iptables v1.8.9 (nf_tables) ← network firewall tables

Usage: iptables -[ACD] chain rule-specification [options]
       iptables -I chain [rulenum] rule-specification [options]
       iptables -R chain rulenum rule-specification [options]
       iptables -D chain rulenum [options]
       iptables -[LS] [chain [rulenum]] [options]
       iptables -[FZ] [chain] [options]
       iptables -[NX] chain
       iptables -E old-chain-name new-chain-name
       iptables -P chain target [options]
       iptables -h (print this help information)

```

Commands:

Either long or short options are allowed.

<code>--append</code>	<code>-A chain</code>	Append to chain
<code>--check</code>	<code>-C chain</code>	Check for the existence of a rule
<code>--delete</code>	<code>-D chain</code>	Delete matching rule from chain
<code>--delete</code>	<code>-D chain rulenum</code>	Delete rule rulenum (1 = first) from chain
<code>--insert</code>	<code>-I chain [rulenum]</code>	Insert in chain as rulenum (default 1=first)
<code>--replace</code>	<code>-R chain rulenum</code>	Replace rule rulenum (1 = first) in chain
<code>--list</code>	<code>-L [chain [rulenum]]</code>	List the rules in a chain or all chains
<code>--list-rules</code>	<code>-S [chain [rulenum]]</code>	Print the rules in a chain or all chains
<code>--flush</code>	<code>-F [chain]</code>	Delete all rules in chain or all chains

Add a firewall rule to Kali

Adds a new rule to the INPUT chain that matches incoming traffic from the IP address 136.160.215.15 and sends it to a Netfilter queue with ID 1 for further processing.

```
(student@kalit01)-[~/dns]  
$ sudo iptables -A INPUT -s 136.160.215.15 -j NFQUEUE --queue-num 1
```

- **sudo** - runs the command as the superuser, which is necessary since **iptables** requires elevated privileges to modify the firewall rules.
- **iptables** – a command-line tool to configure Netfilter, the Linux firewall system.
- **-A INPUT** - appends a new rule to the **INPUT** chain.
- **-s 136.160.215.15** - the source IP address for the rule. The rule will only match traffic that comes from this IP address.
- **-j NFQUEUE** - the target for the rule. The **NFQUEUE** target sends matching packets to a Netfilter queue for further processing.
- **--queue-num 1** - the ID of the Netfilter queue to use. In this case, we're using queue ID 1.



View the newly added firewall rule

```
(student@kalit01)-[~/dns]  
$ sudo iptables -S INPUT  
-P INPUT DROP  
-A INPUT -j ufw-before-logging-input  
-A INPUT -j ufw-before-input  
-A INPUT -j ufw-after-input  
-A INPUT -j ufw-after-logging-input  
-A INPUT -j ufw-reject-input  
-A INPUT -j ufw-track-input  
-A INPUT -s 136.160.215.15/32 -j NFQUEUE --queue-num 1
```

newly added

- -S INPUT: List INPUT chain

```
(student@kalit01)-[~/dns]  
$ sudo iptables -S INPUT | grep "136.160.215.15"  
-A INPUT -s 136.160.215.15/32 -j NFQUEUE --queue-num 1
```

Delete the rule

```
(student@kalit01)-[~/dns]  
$ sudo iptables -D INPUT -s 136.160.215.15 -j NFQUEUE --queue-num 1
```

[sudo] password for student:

```
(student@kalit01)-[~/dns]  
$ sudo iptables -S INPUT | grep "136.160.215.15"
```

```
(student@kalit01)-[~/dns]  
$
```

MUST be the exact same rule
specification

Block a HTTP request

using a rule



Intercept and manipulate network packets

- `netfilter` framework is a set of hooks that can be used to intercept and modify network traffic as it passes through the kernel.
 - `netfilter` framework is a part of the Linux kernel
- `NetfilterQueue` library provides a Python interface to the `netfilter` framework, allowing you to intercept and modify network packets using Python code.



NetfilterQueue

- netfilterQueue is useful for a variety of purposes
 - implementing a firewall,
 - a packet sniffer, or
 - a network intrusion detection system.
- How netfilterQueue works
 - creating a userspace application that sets up a netfilter queue and waits for packets to be intercepted and passed to the userspace.
 - Once a packet is intercepted, it can be modified, dropped, or passed on to its destination, depending on the needs of the application.



Install NetfilterQueue

```
(student@kalit01)-[~/dns]  
$ sudo apt install libnetfilter-queue-dev  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following packages were automatically installed and are no longer required:
```

```
(student@kalit01)-[~/dns]  
$ sudo pip3 install NetfilterQueue  
Collecting NetfilterQueue  
  Using cached NetfilterQueue-1.1.0.tar.gz (90 kB)  
  Installing build dependencies ... done  
  Getting requirements to build wheel ... done
```



A simple traffic monitor that Uses **NetfilterQueue**

Intercept the drop packets based on incoming packets' IP address

```
(student@kalit01)-[~/dns]  
$ wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/firewall/iptables_handle_packets_drop.py
```

`wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/firewall/iptables_handle_packets_drop.py`



Intercept the drop packets if a packet's IP address 136.160.215.194

```
(student@kalit01)-[~/dns]
$ sudo python3 iptables handle packets drop.py 136.160.215.194
Waiting incoming packets...

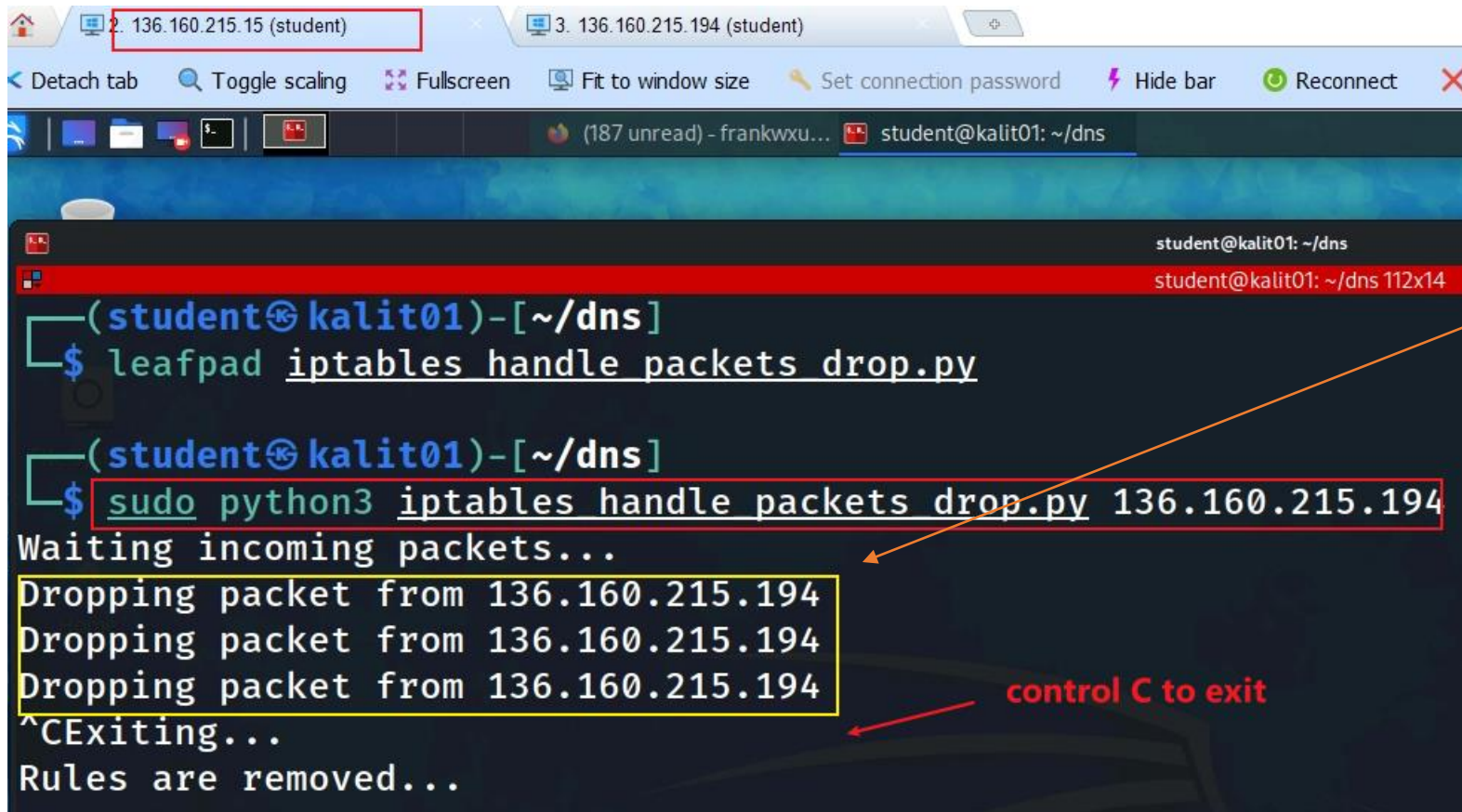
(student@kalit01)-[~/dns]
$ sudo iptables -S INPUT | grep "136.160.215.194"
-A INPUT -s 136.160.215.194/32 -j NFQUEUE --queue-num 1
```

1. Listening and intercepting. It will generate a rule

2. Generating http with ip.src=136.160.215.194 ip.dst=136.160.215.15

```
student@seed-labs01: ~
[03/14/23] student@seed-labs01:~$ curl 136.160.215.15
^C
[03/14/23] student@seed-labs01:~$
```





```
(student@kalit01)-[~/dns]
$ leafpad iptables handle_packets drop.py

(student@kalit01)-[~/dns]
$ sudo python3 iptables handle_packets drop.py 136.160.215.194
Waiting incoming packets...
Dropping packet from 136.160.215.194
Dropping packet from 136.160.215.194
Dropping packet from 136.160.215.194
^CExiting...
Rules are removed...
```

The screenshot shows a terminal window within a web browser. The browser has two tabs: '2. 136.160.215.15 (student)' and '3. 136.160.215.194 (student)'. The terminal is connected to 'student@kalit01: ~/dns'. The user has created a file 'drop.py' using 'leafpad'. Then, they ran 'sudo python3 iptables handle_packets drop.py 136.160.215.194'. The script successfully dropped three packets from the specified IP. The user then pressed Ctrl+C to exit the script, which removed the iptables rules.

3. observe that packets are dropped

```
def iptables_add_rule(ip_address):  
    os.system(f"iptables -A INPUT -s {ip_address} -j NFQUEUE --queue-num 1")
```

bind the rule with the callback function

```
47     q = NetfilterQueue()  
48     q.bind(1, handle_packet)  
49
```

Callback function when a rule is triggered

```
1  def handle_packet(packet):  
2      ip = IP(packet.get_payload())  
3      source_ip = ip.src  
4      print(f"Receiving packet from {source_ip}")  
5  
6      if source_ip == args.source_ip:  
7          print(f"Dropping packet from {source_ip}")  
8          packet.drop()  
9          print("=====")  
0
```

Converting the packet from the `NetfilterQueue` format to the `Scapy` format, which allows us to use the `Scapy` library's functionality to extract information from the packet.

- The `packet` object in this program is an instance of the `netfilterqueue.Packet` class, which is provided by the `NetfilterQueue` library.
- The `NetfilterQueue` library allows you to intercept and modify network packets using a userspace application. The `Packet` class represents a network packet that has been intercepted and passed to the userspace application for processing.
- `packet.get_payload()` method is being used to extract the payload data of the intercepted packet and pass it as an argument to the `IP()` constructor, which creates an `IP` object from the packet data.

Debug: In case failed to bind filters

Show all python processes

```
(student@kalit01)-[~/dns]
$ ps aux | grep python
```

← Show python processes

student	1442	0.0	0.6	375908	27984	?	Sl	Mar02	0:04	/usr/bin/python3	/usr/bin/blueman-applet
student	201457	0.0	2.2	602060	88852	?	Sl	Mar13	0:15	/usr/bin/python3	/usr/bin/terminator
root	214858	0.0	0.1	10224	5068	pts/0	S+	11:56	0:00	sudo python3	iptables_handle_packets_drop.py
root	214861	0.0	0.0	10224	520	pts/2	Ss	11:56	0:00	sudo python3	iptables_handle_packets_drop.py
root	214862	8.1	1.1	56800	47772	pts/2	S+	11:56	0:00	python3	iptables_handle_packets_drop.py 136.1
student	214866	0.0	0.0	6332	2064	pts/1	S+	11:56	0:00	grep --color=auto	python

Kill old python processes

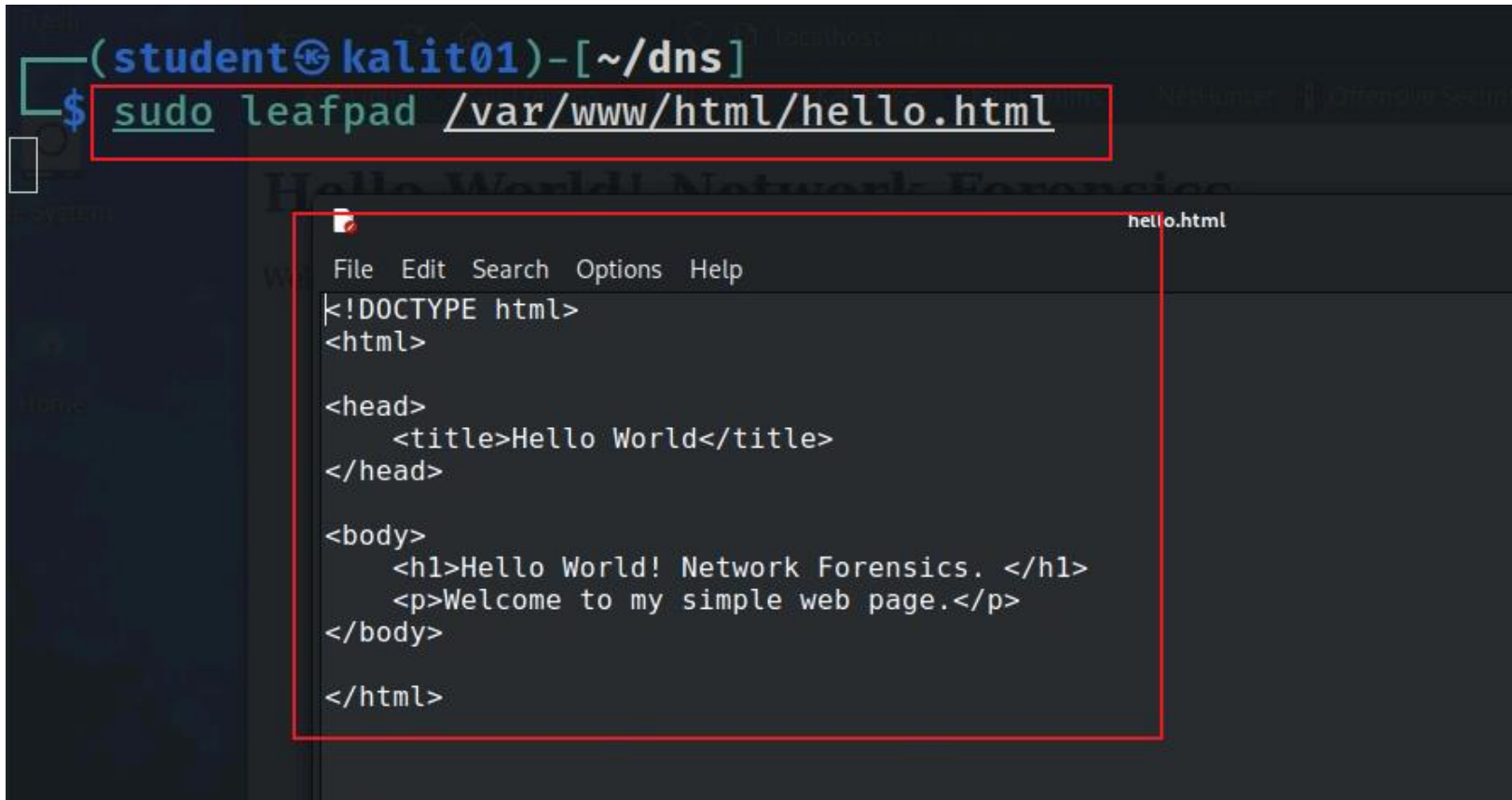
```
(student@kalit01)-[~/dns]
$ kill -9 2124858
```

← kill a Python process if code failed to bind

Block a webpage



Create a web page



The screenshot shows a terminal window with the prompt `(student@kalit01)-[~/dns]`. A red box highlights the command `sudo leafpad /var/www/html/hello.html`. Below the terminal, a text editor window titled `hello.html` is open, showing the following HTML code:

```
File Edit Search Options Help
<!DOCTYPE html>
<html>

<head>
  <title>Hello World</title>
</head>

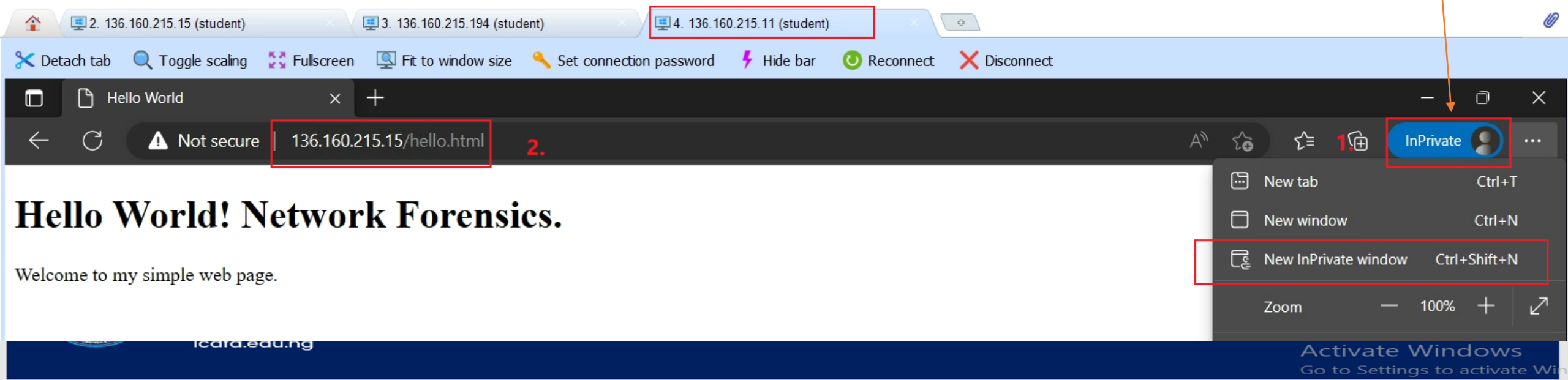
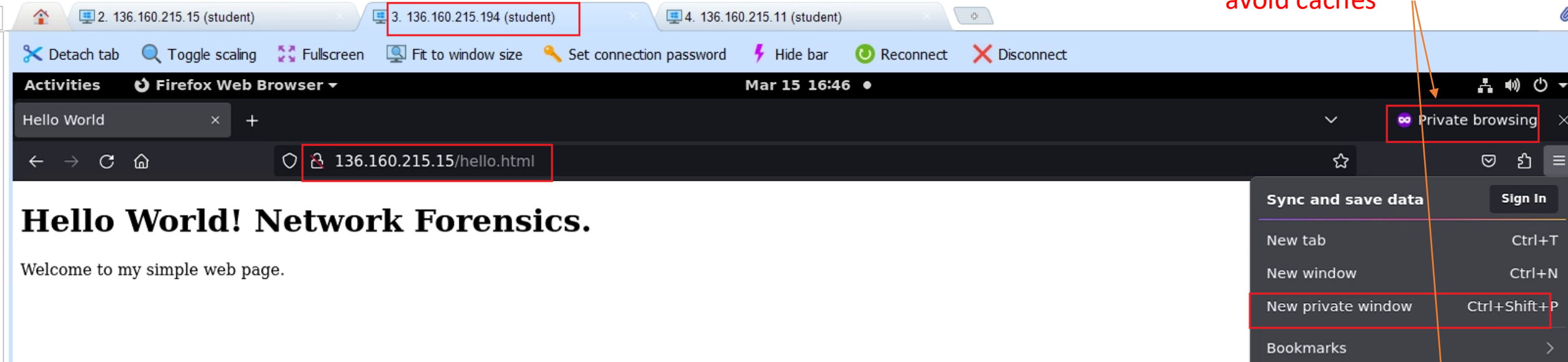
<body>
  <h1>Hello World! Network Forensics. </h1>
  <p>Welcome to my simple web page.</p>
</body>

</html>
```

wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/firewall/hello.html
sudo cp hello.html /var/www/html/.

Before enable a rule of firewall to drop package

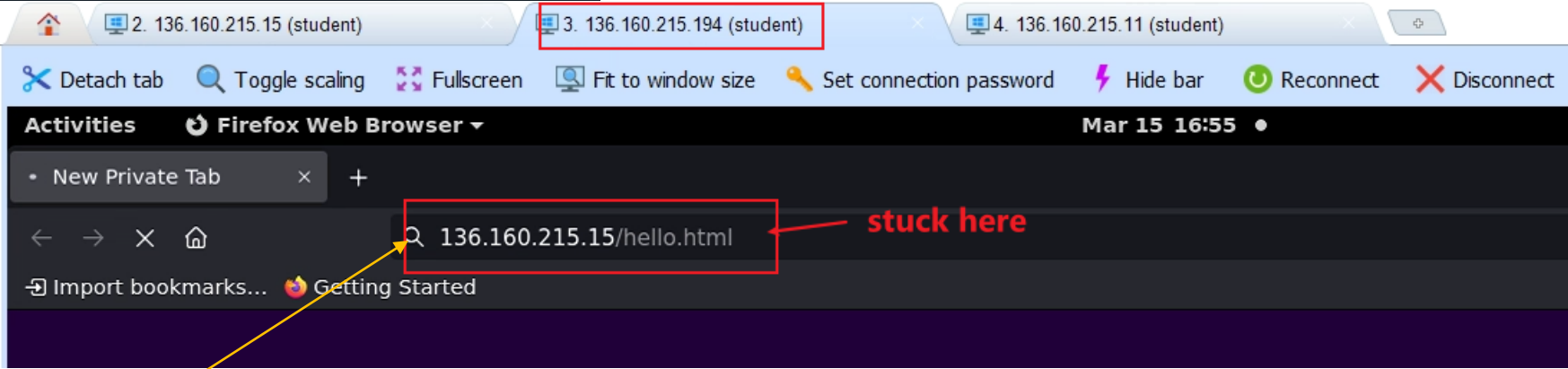
Make sure to use private windows to avoid caches



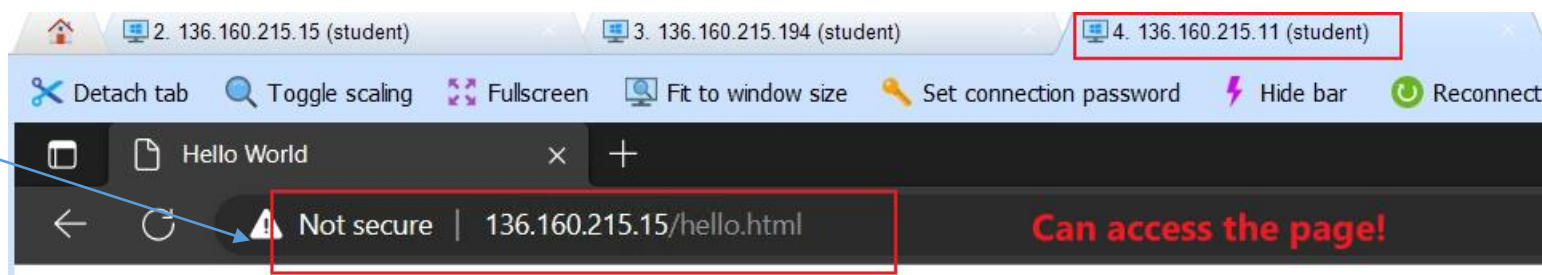

```
student@kalit01: ~/dns
student@kalit01: ~/dns 95x9

(student@kalit01)-[~/dns]
$ sudo python3 iptables handle packets drop.py 136.160.215.194
[sudo] password for student:
Waiting incoming packets...
Receiving packet from 136.160.215.194
Dropping packet from 136.160.215.194
=====}
```

Start the firewall and trigger the rule!



Open a new tab at each host.
.194 is blocked
.15 is allow



Hello World! Network Forensics.

Welcome to my simple web page.

Homework

- Capture traffic generated by `curl` command
- `curl` from a different machine and observe the results

